A

Project Report on

SALTSTACK ARCHITECTURE

Submitted for partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

by

Suhail Ahmed Butt - 24K81A05Q9

Under the Guidance of

Mr. M. Dileep Kumar

Assistant Professor



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ST.MARTIN'S ENGINEERING COLLEGE

UGC AUTONOMOUS

Affiliated to JNTUH, Approved by AICTE

Accredited by NBA & NAAC A+, ISO 9001-2008 Certified.

Dhulapally, Secunderabad-500 100

www.smec.ac.in

St. MARTIN'S ENGINEERING COLLEGE

UGC AUTONOMOUS

Affiliated to JNTUH, Approved by AICTE

Accredited by NBA & NAAC A+, ISO 9001-2008 Certified.

Dhulapally, Secunderabad-500 100

www.smec.ac.in

CERTIFICATE

This is to certify that the project titled **"SALTSTACK ARCHITECTURE"** is being submitted by **Suhail Ahmed Butt (24K81A05Q9)** in fulfillment of the requirement for the award of degree of **BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING** is recorded of bonafide work carried out by them. The result embodied in this report have been verified and found satisfactory.

Signature of Guide
Mr. M. Dileep Kumar
Assistant Professor
Department of CSE

Signature of HOD
Mr. D. Krishna Kishore
Associate Professor & HOD
Department of CSE

Internal Examiner

External Examiner

St. MARTIN'S ENGINEERING COLLEGE

UGC AUTONOMOUS

Affiliated to JNTUH, Approved by AICTE

Accredited by NBA & NAAC A+, ISO 9001-2008 Certified.

Dhulapally, Secunderabad-500 100

www.smec.ac.in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

I, the student of **"Bachelor of Technology in Department of Computer Science & Engineering",** session: 2024 - 2028**, St. Martin's Engineering College**, Dhulapally, Kompally, Secunderabad, hereby declare that the work presented in this project work entitled **"SALTSTACK ARCHITECTURE"** is the outcome of my own bonafide work and is correct to the best of my knowledge and this work has been undertaken taking care of Engineering Ethics. This result embodied in this project report has not been submitted in any university for award of any degree.

Suhail Ahmed Butt (24K81A05Q9)

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompanies the successful completion of any task would be incomplete without the mention of the people who made it possible and whose encouragement and guidance have crowded our efforts with success.

First and foremost, I would like to express our deep sense of gratitude and indebtedness to our College Management for their kind support and permission to use the facilities available in the Institute.

I especially would like to express our deep sense of gratitude and indebtedness to **Prof. Dr. K. Ravindra**, Director & Principal, St. Martin's Engineering College Dhulapally, for permitting us to undertake this project.

I am also thankful to **Mr. D. Krishna Kishore**, Head of the Department, Computer Science and Engineering, St. Martin's Engineering College, Dhulapally, Secunderabad, for his support and guidance throughout our project.

I would like to express my sincere gratitude and indebtedness to my project supervisor **Mr. M. Dileep Kumar,** Assistant Professor, Computer Science and Engineering, St. Martin's Engineering College, Dhulapally, for his support and guidance throughout our project.

Finally, I express thanks to all those who have helped us successfully completing this project. Furthermore, I would like to thank my family and friends for their moral support and encouragement. I express thanks to all those who have helped us in successfully completing the project.

Suhail Ahmed Butt (24K81A05Q9)

CONTENTS

1.ABSTRACT	**01**
2.INTRODUCTION	**02**
3.LITERATURE SURVEY	**03**
4.SYSTEM REQUIREMENTS	**04**
5.FLOW CHART	**05**
6.ALGORITHM	**06**
7.SOURCE CODE	**07**
8.EXPERIMENTAL RESULTS	**10**
9.CONCLUSION	**13**
10.FUTURE ENHANCEMENTS	**14**
11.REFERENCES	**15**

1. ABSTRACT

SaltStack, commonly called Salt, is an open-source platform for remote execution and configuration management of large numbers of servers. Instead of logging in to each machine, an administrator describes the desired state of the infrastructure once, and Salt applies it everywhere. This project studies the architecture of SaltStack: the Salt master, the Salt minions, the ZeroMQ message bus that carries jobs on TCP ports 4505 and 4506, the public-key authentication of minions, and the data sources called grains and pillar that describe each minion. The project also covers states written in YAML and Jinja, the top file that assigns states to minions, and supporting components such as execution modules, the event system, the reactor, returners, salt-ssh, and syndic masters. A working master and minion were set up with Salt 3008.5 on Ubuntu 24.04, and a small web-server configuration for the CampusPortal application was applied with a highstate run. The first run made five changes and the second run made none, which demonstrates idempotence, and targeting by grains was verified. The main objective of this project is to understand how the components of SaltStack work together and to show how a single master can configure systems quickly, consistently, and repeatably in a DevOps environment.

TECHNOLOGY FOR PROSPERITY
UGC AUTONOMOUS

2. INTRODUCTION

Managing servers by hand does not scale. In a traditional environment a system administrator connects to each server, installs packages, edits configuration files, and restarts services. As the number of servers grows, this approach becomes slow and error-prone, and the servers slowly drift away from each other until no two behave in exactly the same way. Configuration management tools solve this problem by describing the required configuration in text files, which can be kept in version control and applied to many machines automatically.

SaltStack follows a master–minion model. A central server called the **Salt master** holds the configuration, and an agent called the **Salt minion** runs on every managed machine. They communicate over a ZeroMQ message bus: the master publishes jobs on TCP port 4505, and the minions send their results back to the master on TCP port 4506. Messages are encrypted with AES, and a minion is trusted only after its public key has been accepted on the master. The desired configuration is written as **states** in YAML with Jinja templating, and a **top file** decides which states apply to which minions. Salt also provides remote execution, so one command can run on many minions at the same time. This project explains these components (Fig. 1) and demonstrates them with a working master and minion that configure a sample CampusPortal web server.

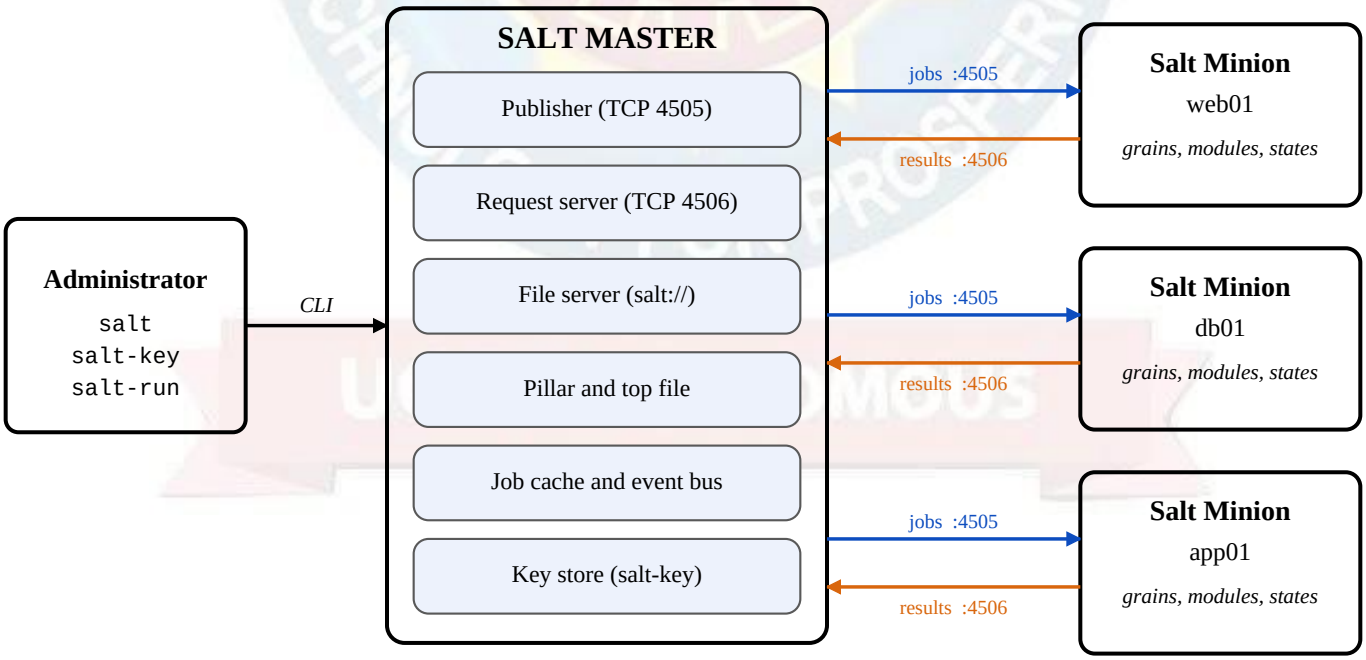


Fig. 1: SaltStack master–minion architecture (minion names are illustrative)

3. LITERATURE SURVEY

Before configuration management tools became common, administrators relied on shell scripts and manual changes. Scripts are easy to write, but they are usually written for one situation, are difficult to repeat safely, and do not describe what the final state of a server should be. Configuration management describes the desired state declaratively so that a tool can bring any machine to that state and keep it there. Early tools such as CFEngine were followed by Puppet, Chef, Salt, and Ansible.

These tools differ mainly in how they reach the managed machine. Puppet and Chef use an agent that periodically pulls its configuration from a server. Ansible needs no agent and pushes tasks over SSH. SaltStack keeps a permanent agent, the minion, connected to the master through a message bus, so the master can push a job to all targeted minions at the same time and collect their replies. Salt can also work without an agent through salt-ssh, and without a master through salt-call --local. Table 1 compares the four tools.

Tool	**Configuration language**	**Model**	**Transport**
Puppet	Puppet DSL	Agent pulls from a server	HTTPS
Chef	Ruby DSL	Agent pulls from a server	HTTPS
Ansible	YAML playbooks	Agentless push	SSH
SaltStack	YAML with Jinja	Master pushes to minions (agentless option)	ZeroMQ (SSH for salt-ssh)

Table 1: Comparison of configuration management tools

SaltStack was created by Thomas S. Hatch and first released in 2011. Its design emphasises speed and scale: the master can target minions by name, by grains, by pillar data, or by compound expressions, and execution modules such as pkg, service, file, and cmd give a uniform command interface on different operating systems. Salt states are plain text files, so they follow the Infrastructure as Code practice of keeping configuration in version control, reviewing it, and applying it repeatedly without side effects. Salt can also serve its files directly from Git repositories through gitfs.

The proposed project concentrates on the architecture of SaltStack. A working master and minion are built, and the authentication, remote execution, grains, pillar, states, and idempotence are verified by experiment.

4. SYSTEM REQUIREMENTS

4.1 Hardware requirements

- Processor: Intel Core i3 or equivalent.
- RAM: Minimum 4 GB; 8 GB recommended (more if several minions run in virtual machines).
- Storage: At least 10 GB of available space.
- Display: Standard laptop or desktop monitor.
- Network: Internet connection for installing Salt packages; TCP ports 4505 and 4506 open between master and minions when they run on different machines.

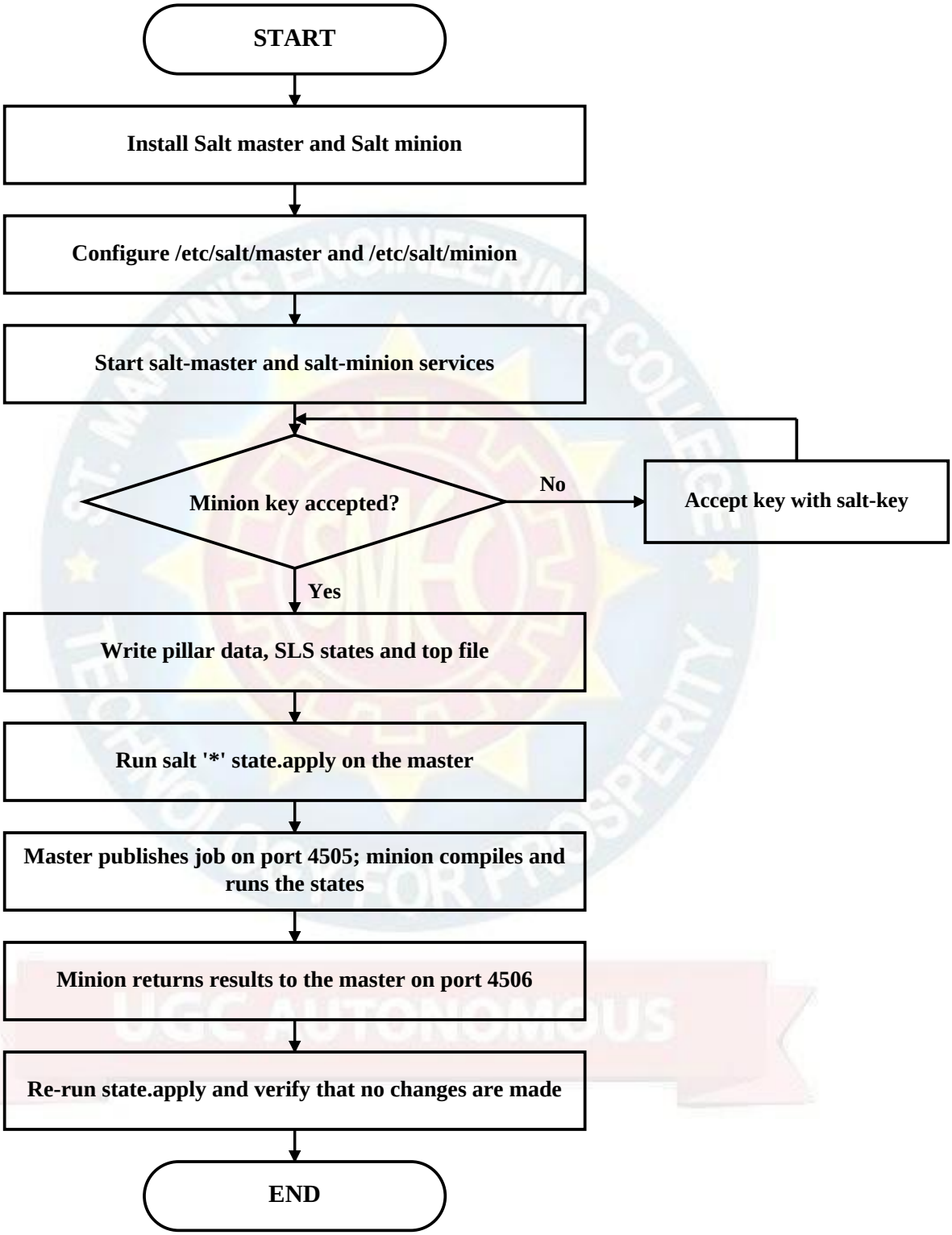
4.2 Software requirements

- Operating System: Ubuntu 22.04/24.04 LTS (used here: Ubuntu 24.04), other Linux, or macOS.
- SaltStack: Salt master and Salt minion, version 3008.5 (Argon) used in this project.
- Configuration languages: YAML for states and pillar; Jinja for templating.
- Runtime: Python is bundled with the Salt packages; no separate installation is required.
- Terminal: Linux terminal with root or sudo access.
- Code Editor: Visual Studio Code or any text editor.
- Version Control: Git and GitHub, if required for storing the project files.
- Optional: VirtualBox or Docker for running several minions.

4.3 Functional requirements

- Install and configure the Salt master and the Salt minion.
- Authenticate the minion by accepting its public key on the master.
- Run remote execution commands on the minion from the master.
- Define grains and pillar data and use them to target and configure the minion.
- Write states with YAML and Jinja and map them to minions with a top file.
- Apply the states with a highstate run and display the results.
- Verify that a repeated run makes no further changes (idempotence).

5. FLOW CHART


START
Install Salt master and Salt minion
Configure /etc/salt/master and /etc/salt/minion
Start salt-master and salt-minion services
Minion key accepted?
No
Accept key with salt-key
Yes
Write pillar data, SLS states and top file
Run salt '*' state.apply on the master
Master publishes job on port 4505; minion compiles and runs the states
Minion returns results to the master on port 4506
Re-run state.apply and verify that no changes are made
END

6. ALGORITHM

1. Start the process.

2. Install Salt on the master system and on the minion system.

3. Configure the master (interface, ports, file_roots, pillar_roots) in /etc/salt/master.

4. Configure the minion (master address, minion id, grains) in /etc/salt/minion.

5. Start the salt-master and salt-minion services.

6. The minion generates an RSA key pair and sends its public key to the master.

7. Accept the minion key with salt-key and check connectivity with test.ping.

8. Define the pillar data, write the state files (SLS), and create the top file.

9. Run salt '*' state.apply; the master publishes the job on port 4505.

10. The minion compiles the states with its grains and pillar and executes them in order.

11. The minion returns the result of every state to the master on port 4506.

12. Run state.apply again and verify that no further changes are made (idempotence).

13. Display the results and stop the process.

7. SOURCE CODE

SaltStack Master–Minion Configuration of a Web Server

1. Master Configuration (/etc/salt/master)

```
# /etc/salt/master
interface: 127.0.0.1
publish_port: 4505
ret_port: 4506
auto_accept: True        # demonstration only; accept keys manually in production

file_roots:
  base:
    - /srv/salt

pillar_roots:
  base:
    - /srv/pillar
```

2. Minion Configuration (/etc/salt/minion)

```
# /etc/salt/minion
master: 127.0.0.1
id: web01

grains:
  role: webserver
  env: production
```

3. Top File (/srv/salt/top.sls)

```
base:
  '*':
    - common
  'role:webserver':
    - match: grain
    - webserver
```

4. Common State (/srv/salt/common/init.sls)

```
install_tree:
  pkg.installed:
    - name: tree

motd_banner:
  file.managed:
    - name: /etc/motd
    - contents: |
        Managed by SaltStack. Minion: {{ grains['id'] }}
        OS: {{ grains['os'] }} {{ grains['osrelease'] }}
```

5. Web Server State (/srv/salt/webserver/init.sls)

```
site_directory:
  file.directory:
    - name: /var/www/{{ pillar['app']['name']|lower }}
    - makedirs: True
```

```
site_index:
  file.managed:
    - name: /var/www/{{ pillar['app']['name']|lower }}/index.html
    - source: salt://webserver/index.html.jinja
    - template: jinja
    - require:
      - file: site_directory

site_config:
  file.managed:
    - name: /etc/{{ pillar['app']['name']|lower }}.conf
    - contents: |
        port={{ pillar['app']['port'] }}
        environment={{ grains['env'] }}
```

6. Jinja Template (/srv/salt/webserver/index.html.jinja)

```
<html>
  <body>
    <h1>{{ pillar['app']['name'] }}</h1>
    <p>Served by {{ grains['id'] }} ({{ grains['role'] }}) on port {{ pillar['app']['port'] }}</p>
  </body>
</html>
```

7. Pillar Files (/srv/pillar/top.sls and /srv/pillar/app.sls)

```
# /srv/pillar/top.sls
base:
  '*':
    - app

# /srv/pillar/app.sls
app:
  name: CampusPortal
  port: 8080
```

8. Commands Used

```
# Install (Ubuntu; add the Salt Project package repository first), then start the services
sudo apt install salt-master salt-minion
sudo systemctl enable --now salt-master salt-minion

# On the master
salt-key -L                          # list minion keys
salt-key -a web01                    # accept one key (auto_accept was enabled for the demo)
salt '*' test.ping
salt '*' grains.item id os osrelease role env
salt '*' pillar.items unmask=True
salt '*' state.apply --state-output=terse
salt -G 'role:webserver' cmd.run 'cat /var/www/campusportal/index.html /etc/campusportal.conf'
```

9. Explanation of the Configuration

1. Master: /etc/salt/master sets the interface and the publish port 4505 and return port 4506, and tells the master where state files (file_roots) and pillar files (pillar_roots) are stored.

2. Minion: /etc/salt/minion gives the master address, a unique minion id, and static grains such as role and env. On first start the minion creates an RSA key pair and offers its public key to the master.

3. Key acceptance: salt-key lists and accepts keys. auto_accept is used only for this demonstration; in production keys should be accepted manually after checking the fingerprint.

4. Grains and pillar: grains are facts about the minion (os, osrelease, role). Pillar is data stored on the master and delivered only to the minions that are targeted in the pillar top file.

5. States: each SLS entry names a state function such as pkg.installed, file.directory, or file.managed. A state describes the end result, not the steps, so it can be applied repeatedly.

6. Jinja: expressions in {{ }} are filled with grains and pillar values before the minion runs the state, which is how one template produces the correct file for each minion.

7. Top file: the top file maps states to minions. Here every minion gets common, and minions with the grain role:webserver also get webserver. The require requisite makes site_index run after site_directory.

8. Targeting: the master can address minions by glob ('*'), by grain (-G), and by other matchers such as pillar, list, regular expression, and compound expressions.

8. EXPERIMENTAL RESULTS

The experiment was carried out on an Ubuntu 24.04 system running Salt 3008.5 (Argon). The Salt master and the minion web01 ran on the same machine and communicated over the loopback address 127.0.0.1, with the master listening on TCP ports 4505 and 4506. All commands were typed on the master. Fig. 2 shows the key list, the connectivity test, and the grains and pillar data of the minion. In this Salt version pillar values are masked in the output by default, so unmask=True was used to display them.

```
root@master:~# salt-key -L
Accepted Keys:
web01
Denied Keys:
Unaccepted Keys:
Rejected Keys:

root@master:~# salt '*' test.ping
web01:
    True

root@master:~# salt '*' grains.item id os osrelease role env
web01:
    ----------
    env:
        production
    id:
        web01
    os:
        Ubuntu
    osrelease:
        24.04
    role:
        webserver

root@master:~# salt '*' pillar.items
web01:
    ----------
    app:
        ----------
        name:
            **********
        port:
            **********

root@master:~# salt '*' pillar.items unmask=True
web01:
    ----------
    app:
        ----------
        name:
            CampusPortal
        port:
            8080
```

Fig. 2: Key list, connectivity test, grains and pillar data

Fig. 3 shows the first highstate run, in which all five states were applied and reported as Changed (the installation of the package tree took most of the 11.527 s run time), followed by a second run in which every state was already in the required condition and was reported as Clean.

```
root@master:~# salt '*' state.apply --state-output=terse
web01:
  Name: tree - Function: pkg.installed - Result: Changed -
      Started: 06:02:18.233662 - Duration: 11483.415 ms
  Name: /etc/motd - Function: file.managed - Result: Changed -
      Started: 06:02:29.723582 - Duration: 11.354 ms
  Name: /var/www/campusportal - Function: file.directory - Result:
      Changed - Started: 06:02:29.735035 - Duration: 0.558 ms
  Name: /var/www/campusportal/index.html - Function: file.managed
      - Result: Changed - Started: 06:02:29.735767 - Duration: 25.482
      ms
  Name: /etc/campusportal.conf - Function: file.managed - Result:
      Changed - Started: 06:02:29.761349 - Duration: 5.975 ms
Summary for web01
------------
Succeeded: 5 (changed=5)
Failed:    0
------------
Total states run:     5
Total run time:  11.527 s

root@master:~# salt '*' state.apply --state-output=terse
web01:
  Name: tree - Function: pkg.installed - Result: Clean - Started:
      06:02:35.361178 - Duration: 55.216 ms
  Name: /etc/motd - Function: file.managed - Result: Clean -
      Started: 06:02:35.418658 - Duration: 15.415 ms
  Name: /var/www/campusportal - Function: file.directory - Result:
      Clean - Started: 06:02:35.434174 - Duration: 0.374 ms
  Name: /var/www/campusportal/index.html - Function: file.managed
      - Result: Clean - Started: 06:02:35.437572 - Duration: 9.5 ms
  Name: /etc/campusportal.conf - Function: file.managed - Result:
      Clean - Started: 06:02:35.447168 - Duration: 1.675 ms
Summary for web01
------------
Succeeded: 5
Failed:    0
------------
Total states run:     5
Total run time:  82.180 ms
```

Fig. 3: First and second highstate runs (state.apply)

The second run took 82 ms against 11.527 s for the first because no work was needed. This confirms that Salt states describe a target condition and are idempotent. Only one minion on one machine was used, so the experiment verifies the architecture and behaviour but does not measure performance with many minions.

Fig. 4 verifies the result on the minion. The index page and the configuration file were rendered from the Jinja template with pillar and grain values, the package was installed, the message of the day was written, and the master and minion sockets were found listening on ports 4505 and 4506.

```
root@master:~# salt -G 'role:webserver' cmd.run 'cat
    /var/www/campusportal/index.html /etc/campusportal.conf'
web01:
    <html>
      <body>
        <h1>CampusPortal</h1>
        <p>Served by web01 (webserver) on port 8080</p>
      </body>
    </html>
    port=8080
    environment=production

root@master:~# salt '*' pkg.version tree
web01:
    2.1.1-2ubuntu3.24.04.2

root@master:~# salt '*' cmd.run 'cat /etc/motd'
web01:
    Managed by SaltStack. Minion: web01
    OS: Ubuntu 24.04

root@master:~# cat /proc/net/tcp  (filtered for LISTEN sockets on
    4505/4506)
Proto  Local Address      State
tcp    127.0.0.1:4505       LISTEN
tcp    127.0.0.1:4506       LISTEN
```

Fig. 4: Verification of the configured minion

Check	**Command**	**Result**
Minion key accepted	salt-key -L	web01 listed under Accepted Keys
Connectivity	test.ping	True
Grains	grains.item	role: webserver, env: production
Pillar delivery	pillar.items	app name CampusPortal, port 8080
First highstate	state.apply	5 succeeded (changed=5), 0 failed
Second highstate	state.apply	5 succeeded, none changed (82 ms)
Rendered files	cmd.run	Values from pillar and grains present
Total		**7 checks passed, 0 failed**

9. CONCLUSION

The project “SaltStack Architecture” explains how SaltStack manages systems through a master and minions that communicate over an encrypted ZeroMQ message bus. A working master and minion were built and tested: the minion key was accepted, a remote command was executed, grains and pillar data were delivered, and a web-server configuration written as YAML and Jinja states was applied with one command. A repeated run made no changes, which shows that the states are idempotent.

The project gives practical knowledge of remote execution, configuration management, templating, targeting, and Infrastructure as Code in a DevOps environment. The experiment used a single machine, so scaling to many minions, multiple masters, and high availability were studied only as concepts. In the future the project can be extended with more minions, event-driven automation with the reactor, and integration with CI/CD pipelines to make the infrastructure more reliable, secure, and efficient.

10. FUTURE ENHANCEMENTS

1.Multiple Minions: Add several minions in virtual machines or containers and compare targeting with grains, pillar, and compound matchers.

2.Reactor and Beacons: Use the event system so that a change on a minion, such as a stopped service, automatically triggers a corrective state.

3.Agentless Management: Use salt-ssh to manage machines that cannot run a minion.

4.Gitfs and CI/CD: Serve states from a Git repository and test and apply them automatically with GitHub Actions or Jenkins.

5.Scalability: Study syndic masters and multi-master set-ups for large or geographically separated infrastructures.

6.Security Hardening: Disable auto_accept, verify key fingerprints, restrict access to ports 4505 and 4506, and keep secrets in pillar or an external secret store.

7.Salt API and Dashboard: Expose Salt through its API and build a dashboard that displays job results and minion status.

11. REFERENCES.

1.Salt Project Documentation. https://docs.saltproject.io/en/latest/

2.Salt User Guide. https://docs.saltproject.io/salt/user-guide/en/latest/

3.Salt Documentation. Targeting Minions. https://docs.saltproject.io/en/latest/topics/targeting/index.html

4.Salt Documentation. Pillar. https://docs.saltproject.io/en/latest/topics/pillar/index.html

5.ZeroMQ Documentation. https://zeromq.org/

6.Jinja Documentation. https://jinja.palletsprojects.com/

ST. ANN'S ENGINEERING COLLEGE
SMK
TECHNOLOGY FOR PROSPERITY
UGC AUTONOMOUS